

Switch-based Schemes for TCP Performance Enhancement in Data Centers: Design, Synthesis and Evaluation

Ahmed M. Abdelmoniem and Brahim Bensaou
Department of Computer Science and Engineering
The Hong Kong University of Science and Technology
Clear Water Bay, Hong Kong
{amas, brahim}@cse.ust.hk

June 28, 2017

Abstract

In this report, we explore the prospects and challenges of switched-based data center AQM schemes presented in our previous papers [2, 1, 8]. In particular, we showcase that such schemes are feasible solution for operators to consider. To this end, inspired by the appealing results of RWNDQ ([2]), IQM ([1]) and HSCC ([8]), we propose, design and prototype the proposed switch-assisted congestion resolution mechanisms in the NetFPGA platform [10]. These mechanisms try to address and resolve wide spectrum of congestion causes in data centers. First, we propose on NetFPGA platform a switch-prototype called “FairSwitch” that can fairly allocate its buffer space among competing flows regardless of their type (i.e., mice or elephant). Then, we focus on Incast events and propose another switch-prototype named “IncastGuard” that monitor certain signals to anticipate Incast beforehand ultimately avoiding possible buffer-overflow. Finally, we resolve the impairment of short TCP cycles in datacenters and present a switch-prototype called “HystSwitch” that enhances TCP performance by stretching out the loss cycles. The three mechanisms do not require any changes to the TCP protocol at the sender nor at the receiver and are shown to be effective in achieving their design goals via deploying them on NetFPGA card and running experiments in a small-scale cluster testbed.

1 Introduction

DCN operators use commodity Ethernet switches with small buffers for interconnecting the servers mainly for their low cost and ease of deployment. Typically, the in-network switches are under the control of the cloud (or datacenter) operator. These switches have certain lifetime (and even worse they frequently fail before that) after-which they need to be replaced just like any other hardware equipment. Cloud operators are then forced to buy new (possibly upgraded models) switching devices to replace old or failed ones. Hence, we believe that solutions involving very simple modifications of the switching chips is quite viable and appealing to cloud operators. Switching chip manufacturers would be motivated by marketing a datacenter efficient switching devices that involve minimal hardware/software updates yet they provide significant performance gains.

Our motivation towards building a proof-of-concept prototype is to provide a show-case for our proposed AQMs to the industry and potential switch fabric manufacturers. The prototype would also allow us to experiment with our AQM schemes in a realistic data center like environment. It would also allow for macro and micro benchmarking our schemes to establish its complexity, performance and overhead. This task was eased by the introduction of the 4-ports NetFPGA 1Gbps and 10Gbps cards. This project was initiated by Stanford University and the initial design and model of NetFPGA board was introduced in 2007. The aim of this project is to facilitate research and prototyping of networking protocols for routing and switching devices [7]. Hence, in this report, we introduced our prototyping procedure, discussion of practical findings and a summary of possible development methods in NetFPGA platform. Our sole aim of this report as a computer scientist is to ease the adoption of hardware prototyping for fellow computer scientists. Typically, computer science researchers lean away from such tedious task involving implementations using a non-familiar hardware languages.

Since switches are networking elements that are under-control of DC operators, they are typically a viable update option for DC operators. Hence, we present the prototyping of three hardware switch AQM designs (i.e., RWNDQ in [2], IQM in [1] and HSCC in [8]). The prototyped switches should possess the following features: (F1) Improve latency-sensitive applications Flow Completion Time (FCT) and mitigate incast; (F2) Elephant flows can enjoy high sustained throughput in a work-conservative manner; (F3) No modifications are imposed on the existing TCP protocol stack or anything that is controlled by the tenant¹;

¹If changes to existing systems are needed (i.e., inevitable) then they must be in network de-

To this end, we present “FairSwitch” (resembling RWNDQ, the equal-share allocation scheme) and “IncastGuard” (resembling IQM, the proactive Incast detection scheme) and “HystSwitch” (resembling HSCC, the hysteresis-based AQM scheme). In FairSwitch, the switch actively enforces rate adjustments in a fair manner to regulate the queue at a certain target occupancy. While in IncastGuard, the switch intervenes only temporarily in order to pace down buffer-greedy elephant flows to help short-lived mice flows pass the bottleneck link with minimal congestion and least delays as possible. HystSwitch leverages the established control theory to improve the duration of TCP loss cycles in data centers. This is achieved by enforcing a hysteresis-based switching between slow and fast arrival rates. All of these designs leverage TCP flow control mechanism to control TCP flow rates by adjusting the “advertised” window field without intervening with their current “congestion” window update function.

1.1 The NetFPGA Hardware Platform

The NetFPGA is an open platform for gigabit Ethernet switching and routing that has been developed at Stanford University. Stanford University started the design and development of NetFPGA board in 2007 to facilitate research and prototyping of networking protocols for routers and switches [7]. NetFPGA is a complete network hardware implemented on a FPGA along with 4 network ports. It is mostly used to build and test new protocols and network functions. The NetFPGA-1G board is equipped with Xilinx Virtex-II Pro 50 FPGA chip, 4 Gigabit Ethernet networking ports, Cypress 4.5 MB SRAM, Micron 64 MB DDR2 DRAM, and standard PCI Card [10]. The board comes with multiple online open-source reference projects all written in Verilog and many other contributed open-source projects published by researchers. The NetFPGA platform has gained a lot of attention and was improved over the years to keep up with the new hardware configurations. At the time of this writing, the card we used is deprecated and replaced with 3 other flavors (i.e., SUME, CML and 10Gbps) for 1Gbps and 10Gbps networks with various pricing and educational discounts². All of our switch designs relied on the reference switch model whose bitfile and source code are provided with the board.

vices and/or hypervisors that are fully under the control of the DCN operator

²NetFPGA website is available at <http://www.netfpga.org>

1.2 The Reference Router and Switch Design

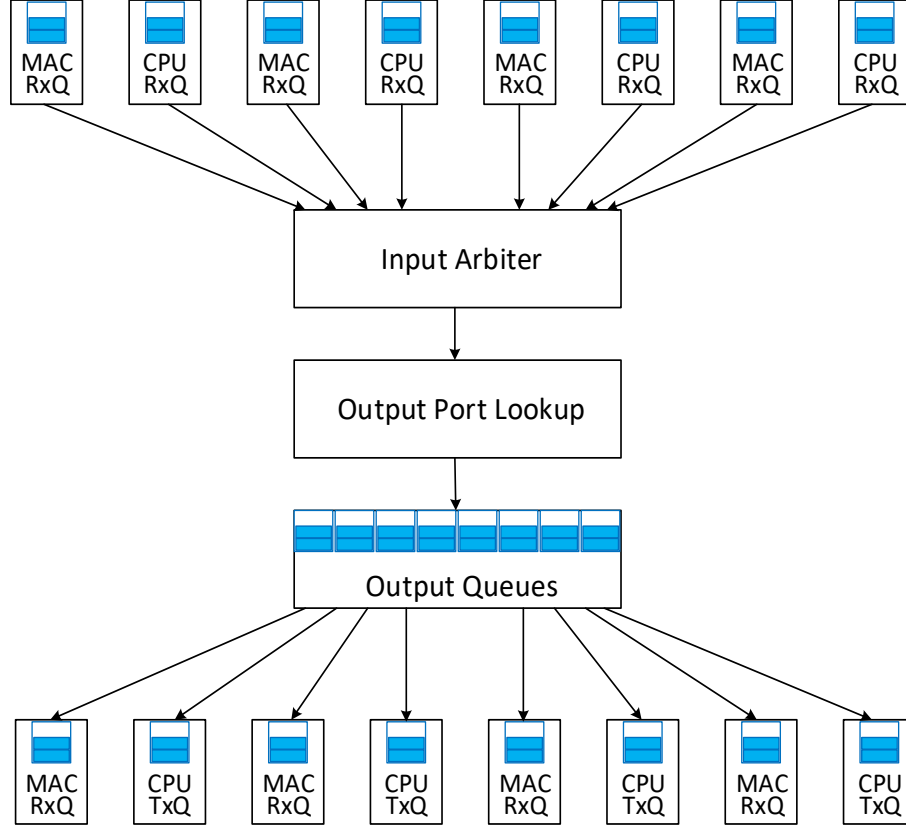


Figure 1: *NetFPGA-based Modular Pipelined Design*

The reference IPv4 router and the switch designs are part of the NetFPGA base package. They consist of a number of verliog modules that are all connected together in a pipelined manner. Typically, they are organized in a way that allows the board to operate at the speed of 125 MHz (i.e., 8 nano-seconds per clock cycle) [7]. For the purpose of this report, first, we need to introduce the modular design of the NetFPGA to fully grasp the operational internals of it. Afterwards, this design is modified to implement our proposed switch AQM designs. Figure 1 shows the pipelined structure of on which all the NetFPGA reference designs are built. The design features 4 physical (1 Gbps or 10 Gbps) Ethernet ports controlled

by the FPGA board. Each of port has an ingress or egress direction which are represented as a TX port (egress) and a RX port (ingress) for the switch. The FPGA exposes another 4 “logical” DMA ports (i.e., they are connected to the CPU of the hosting PC). Each logical port relates to each physical port to act as the channel between the physical port and the host on which the PCI card resides. In cases when the reference NIC is not in use, the reference router/switch transfers the incoming packet to the respective DMA port and then to the CPU for processing whenever it un-able to process or handle it.

Most of the major processing in the NetFPGA is carried out by 3 other modules that constitutes the reference design. The first module is the input arbiter which uses a round robin arbiter to pick up packets from the input queues and forwards them to the corresponding FIFO of the output port Lookup module. The output lookup module does the packet processing, namely, parsing the Ethernet header, the IP/LPM lookup, and destination filtering and finally, modifying the Ethernet headers. It then hands the packet to the “output queues” module that initially stores the packet in the SRAM and then hands it to the corresponding physical port when it is available for transmission. In addition, between all the modules, the design employs an input FIFO in the pipeline between two successive modules (i.e., output lines of the previous to the input lines of next). This input FIFO is used for pipelining the design. The current module reads the data from the input FIFO as and when required and processes it.

1.3 An End-host Helper Module for Window Scaling

All switch designs discussed in the following sections rely on a scale factor to rescale the used window for updating the ACKs when the receive window *Rwnd* is updated. TCP specification [6] states that the three-byte Scale option may be sent in all packets or only in a SYN segment by each TCP end-point to let its peer know what factor it uses for its own window value scaling. TCP implementations in most popular OSes including linux adopt the latter approach to save overhead and wasted bandwidth of the former approach. The scaling may be unnecessary for networks with Bandwidth-Delay Product (BDP) of 31.25 Kbyte (i.e., C=1 Gbps and RTT=250 μ s). However, with the introduction of high speed links of 40 Gbps (i.e., BDP=1.25 Mbyte) and 100 Gbps (i.e., BDP=3.125 Mbyte), the scaling factor becomes necessary to utilize the bandwidth effectively. To this end and to avoid any expensive flow-level state tracking at the switch, we propose the adoption of a light-weight end-host (hypervisor) shim-layer to explicitly send scaling factor with outgoing ACKs. It extracts and stores from outgoing SYN and

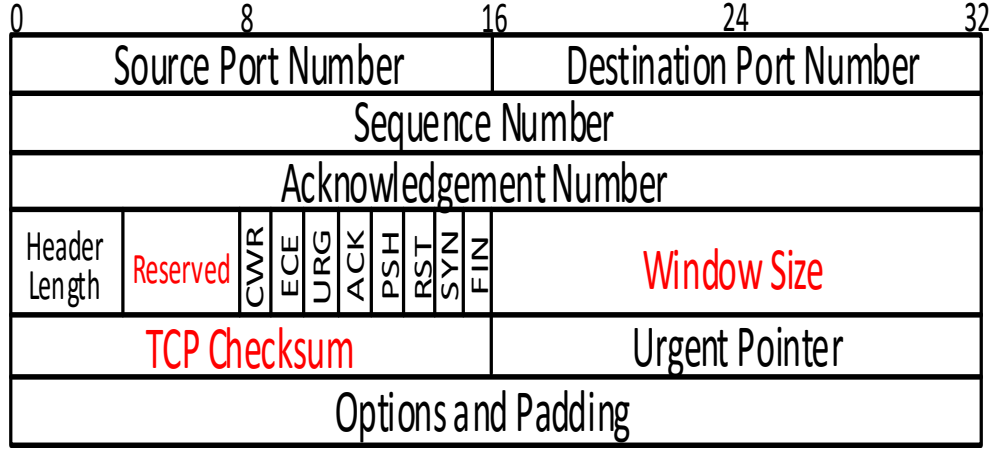


Figure 2: *TCP headers manipulated our systems*

SYN-ACK packets the advertised scaling factor (i.e., within the window scaling option) for each established TCP flow. The system, at connection-setup, hashes the flows into a hash-table. It identify the flows through the 4-tuple (i.e., source IP, dest. IP, source port and dest. port) as the key and stores the window scale factor as the entry value. Flow entries are cleared from the table when a connection is closed (i.e., FIN is sent out). Then, as shown in Fig 2, the module writes the scale factor for all outgoing ACK packets in the 4-bit reserved field of TCP headers (alternatively, by using 4-bits of the receive window field and using the remaining 12 bits for window values). The used reserved bits is cleared after their usage by the FairSwitch switch to avoid packet being dropped by destination due to invalid TCP check-sum value which avoids the need for recalculating TCP checksum at the end-host and the switch. Typically, the module resides right above the NIC driver for a non-virtualized setups, right below the hypervisor to support VMs in cloud datacenters or implemented in the data-path of the NIC (e.g., FPGA-based NICs) if applicable. Hence, this placement does not touch any network stack implementation of the guest OS, making it readily deployable in production datacenters.

In the remainder of this report, we present our design and prototyping in a hardware-centric manner. To this end, FairSwitch, IncastGuard and HystSwitch mechanisms are presented and discussed in Section 2, 3 and 4, respectively.

2 FairSwitch for Switch-based Fair Share Allocation

In this section, we discuss the hardware design aspects of FairSwitch in NetFPGA platform. FairSwitch is a hardware realization of a simple switch AQM mechanism that is based on the following ideas: 1) We conclude that to govern the switch buffer usage among competing (distributed) TCP flows, a mechanism has to intervene to equally share the limited buffer space among these flows; 2) To be able to do so, the switches are the ones that have the most accurate information on the nature of congestion, therefore FairSwitch adopts an AQM namely “RWNDQ” to control buffer occupancy at a certain level; 3) To avoid modifying existing protocols at the sender and receiver, FairSwitch must rely on universally accepted mechanisms for congestion notification. The switch can be involved in actively rewriting the *Rwnd* field as a means to throttling the sender’s rate of each connection. 4) How frequently should such rewriting take place? in this case, the switch rewrites *Rwnd* for every ACK on the backward path to the senders.

Based on these ideas, FairSwitch is a hardware realization of the RWNDQ algorithm. RWNDQ is meant to equalize the *Rwnd* field among TCP flows to achieve a target *Q_Target* occupancy that can reduce the latency for mice TCP flows and overcome the burstiness of elephant flows.

2.1 NetFPGA Design and Implementation

Hardware implementation of FairSwitch was developed on the NetFPGA-1G card using “Verliog” hardware description language. For FairSwitch, we have chosen to build it on top of the reference switch project by integrating our module into the “output queues” module. The required modifications were realized through 560 lines of Verliog code including declarations.

2.1.1 NetFPGA FairSwitch module

The main design is implemented in “Fair_Switch” module which is instantiated within the top “output_queues” module as shown in Fig 10. The Fair_Switch module instantiates a number of helper modules namely “avg_32”, “div_32”, “TCP_checksum” and “ECN_mark” for different data processing. The main inputs to the module is the incoming packet (i.e., *in_data*), control information added by NetFPGA module (i.e., *in_ctrl*) and data ready signal (i.e., *in_wr*) all coming from

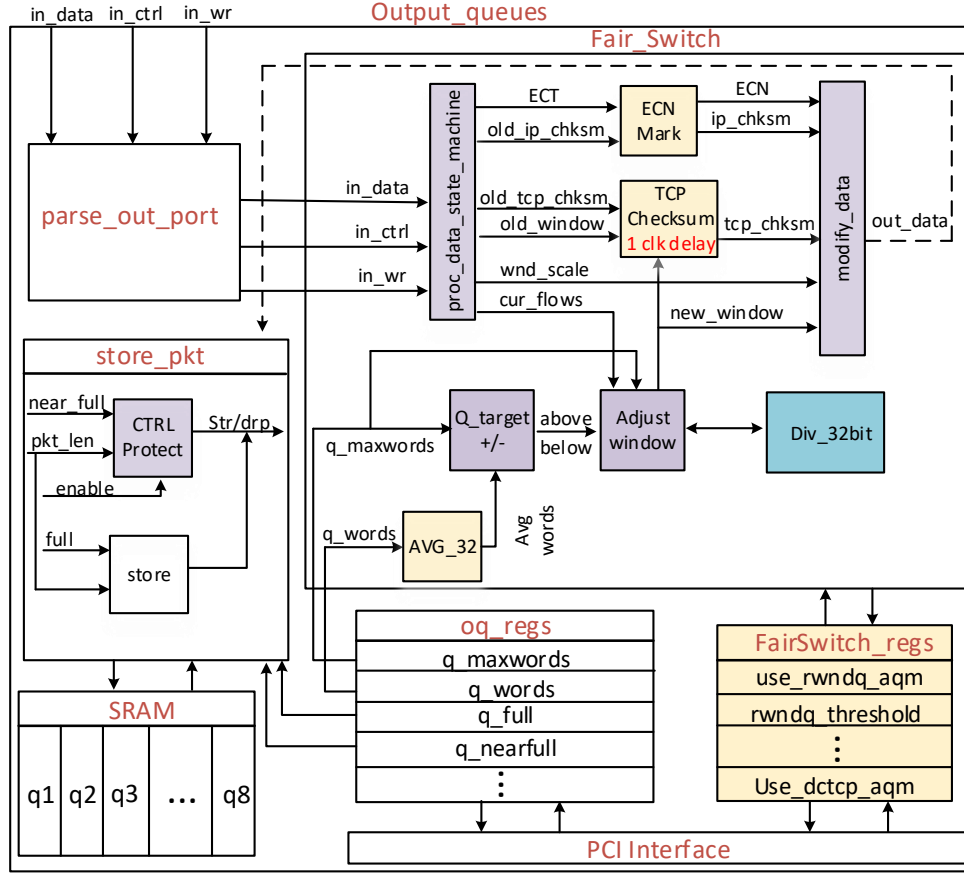


Figure 3: FairSwitch NetFPGA-based system design

the output_queues module and the main outputs are the (un)modified packet information (i.e., out_data, out_ctrl and out_wr) to be written into a FIFO for later storage in the destination queue's SRAM space. Other worth-mentioning inputs are q_maxwords and q_words coming from oq_regs module which is the allocated SRAM space for the destination queue and the number of words currently used by stored packets.

The module keeps track of separate per-output-queue counters for SYN/SYN-ACK, FIN/FIN-ACK and RST packets seen going through the queue. This information can be extracted by applying the state machine on in_data to process the headers. Typically, when line in_wr is active and line in_ctrl is disabled then the word under processing is part of either the header of the packet or the meta-data

added by NetFPGA. The current number of active flows is increased by one for the incoming and outgoing queue, each time a SYN-ACK is received. Whenever a FIN-ACK/RST is received, this number is decreased by one for the outgoing direction of the FIN-ACK. At any time, the absolute difference between SYNs and FINs represents the number of currently active flows. This value is reset whenever no data packets is received from any of the active flows for enough period of time (i.e., 1 sec). The decision that the safe threshold (i.e., target queue occupancy) has been exceeded is evaluated for every output-queue. This is achieved by current average number of occupied words, and maximum number of words in the queue right shifted by the target threshold³. If the queue occupancy is above or below the threshold, the current queue local window is readjusted following the RWNDQ Algorithm ???. Typically, new window value is re-calculated, whenever the variable `cur_flows` is updated due to change in the number of currently ongoing connections. Then, in this case, the target occupancy (i.e., `q_maxwords` $\geq$ `rwndq_threhshold`) is divided by `cur_flows` to calculate the new window value. And then, the window is adjusted up and down to maintain the target threshold.

On the arrival of each ACK, before it is forwarded to a certain output queue (port), the receive window field of TCP header is updated (only if it is less than the current value in TCP header) with the newly calculated window. The new window is right shifted by the “window scale” value which can be extracted out from the reserved-bits field shown in Figure 2. Note that, the `rwndq_threhshold` should be set in a way to preserve an almost empty queue for absorbing elephants and large enough to keeping the link busy by elephants.

2.1.2 FairSwitch helper modules

FairSwitch module relies on the following helper modules instantiated within to perform various functions:

1. **AVG_32:** This module is used to calculate a running average of the number of words used per-output-queue. This is accomplished by averaging `q_words` over the last 32-samples taken every `avg_queue_time`. `avg_queue_time` is a customizable timer (default set to $48\mu s$ which equals the transmission time of four 1500 Bytes sized packets over 1 Gbps link).
2. **TCP_checksum:** This module calculates the new TCP checksum using the incremental update [11] method shown in line 13 of Algorithm ???. It takes the window, new window and current checksum as input and outputs the

³Right shifting by x is equivalent to multiplying by 2^{-x}

new checksum value in the next clock cycle. For this reason, the `in_data` needs to be delayed by one clock cycle to be able to write it into the correct word of the `out_data`.

3. **ECN_mark:** This module implements the DCTCP's AQM marking mechanism, in which packets are marked whenever a customizable fraction (default is 25% which is close to the recommended 20% value) of the maximum queue occupancy has been exceeded. It also recalculates a new IP checksum using ECT (01 or 10), ECN (11) and the old checksum value using the incremental update [11] method. In this case however, no extra delay is needed as IP checksum is one word past the TOS field⁴.
4. **DIV_32bit:** This module is used for the window calculation by dividing the left shifted (i.e., `gg`) value `q_maxwords` by `rwndq_threshold` over `cur_flows` (i.e., setting a window of equal buffer share among flows). This divider is generated from Xilinx IP cores library and an instance of it is invoked. The division process takes 18 clock-cycles to complete but there is no need to add extra 18 cycle delay to `in_data`. This is because if this feature is used, the newly calculated window is updated with SYN/FIN arrivals and ACKs arrive with $\approx$ half RTT from SYN's arrival in the backward path.
5. **FairSwitch_regs:** This module is instantiated in the `output_queues` module to store all FairSwitch related parameters (stored in registers) for usage by various module functions. It also communicates with the Peripheral Component Interconnect (PCI) interface for reading and/or setting the FairSwitch related parameter values.

Figure 3 also shows how FairSwitch module interacts with the other modules within `output_queues` main module. Specifically, after the packet is processed by `output_port_lookup` module to determine the outgoing port, it is processed and stored by `output_queues` for later transmission by the `input_arbiter` (or `output_arbiter` in this case) as illustrated in Figure1. The destination port information is first parsed and extracted by `parse_out_port` module from the control headers which is added by `output_port_lookup`. Then the packet is processed by FairSwitch module and its output (i.e., `out_data`) is sent to `store_pkt` module for storing it in the SRAM or dropping it otherwise.

Finally, to assess the complexity of our module in terms of resource usage, Table 1 highlights the increase in the resources between the reference switch design

⁴NetFPGA processes data in words where each word consists of 64-bits (i.e., 8 bytes) takes 1 clock cycle.

Table 1: Comparison of resource usage between the reference switch and FairSwitch

	Reference Switch		FairSwitch		NetFPGA Total
	Total	Percentage	Total	Percentage	
Slices	9605	40%	10851	45%	23616
Slice Flip Flops	8364	17%	9419	19%	47323
4 input LUTs	14067	29%	15985	33%	47232
BRAMs	22	9%	22	9%	232
Bonded IOBs	360	52%	360	52%	692
CGLKs	8	50%	8	50%	16
DCM	6	75%	6	75%	8

and the modified one that includes FairSwitch module.

2.2 Deployment of the IncastGuard Prototype

We inspect the performance of the hardware prototype of RWNDQ namely “FairSwitch” switch as well as its end-host helper module which are presented in Appendix ???. Figure 4 shows the deployment of FairSwitch in a datacenter or cluster setup and how would its components interact with each other. The end-host module tracks the scaling factor used by local communicating end-points and explicitly append this information only to outgoing ACKs of the corresponding flow. The FairSwitch updates receive window of all incoming ACKs to assign a fair share of output port buffer. The scale factor in packet header is used to rescale the new window so that it can be interpreted correctly by the ACK receiving end-point.

Figure 4 shows that the end-host module tracks the scaling factor used by local communicating end-points and explicitly append this information only to outgoing ACKs of the corresponding flow. The switch monitors the queue occupancy of the per-port output queue and the arrival of special packets such as SYN-ACK/FIN-ACK. The switch actively calculates a local per-port window value based on the amount of deviation from the target queue occupancy. The window values is re-distributed each time a new flow is observed or an existing flow leaves the network. The switch is also continuously rewriting the receive window field of all incoming ACKs if the local value is less than the value in the header. The attached window scaling value is used to rescale the new window so that it can be interpreted correctly by the ACK receiving end-point.

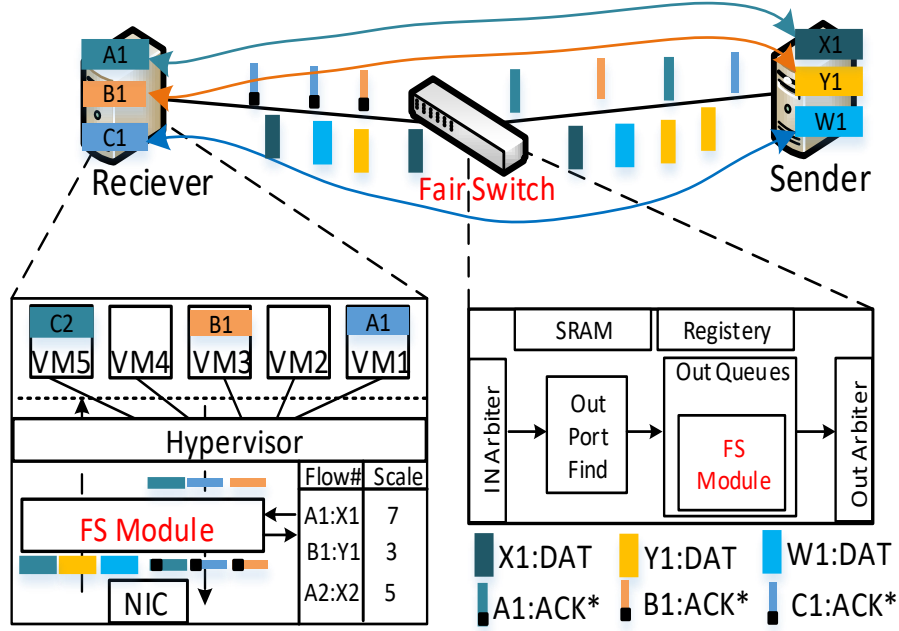


Figure 4: *FairSwitch System: consists of an end-host module that attaches scaling value to ACKs and FairSwitch which performs equal fair share allocation.*

In the next set of experiments, we deploy the NetFPGA-based FairSwitch in a small-scale real-testbed. The testbed consists of 28 virtual servers each associated with a physical dedicated 1GB Network Interface Card (NIC) and running on 7 high performance Dell PowerEdge R320 machines. The machines are equipped with Intel Xenon E5-2430 6-cores CPU, 32 GBytes of RAM and Intel I350 server-grade 1 Gb/s quad-port NIC. As shown in Figure 5, The servers are organized into 4 racks (each rack is a subnet) and connected via 4 non-blocking Top-of-Rack (ToR) switches and the NetFPGA FairSwitch serves as the core switch of the network. The 4 racks are divided into (rack 1, 2 and 3) as senders and rack 4 as the receiver. Each 7 out of the 28 ports belonging to the same subnet are connected to one of the non-blocking ToR switches through 1 Gb/s links. The servers are installed with Ubuntu Server 14.04 LTS upgraded to kernel version (3.18) which has by default the implementation of DCTCP [3], Cubic and New Reno congestion control mechanisms. Finally, we also run on all end-hosts the FairSwitch end-host module as a loadable NetFilter [9] Linux Kernel module.

For experimentation purposes, the machines are installed with the iperf program [5] for creating elephant flows and the Apache web server hosting a single

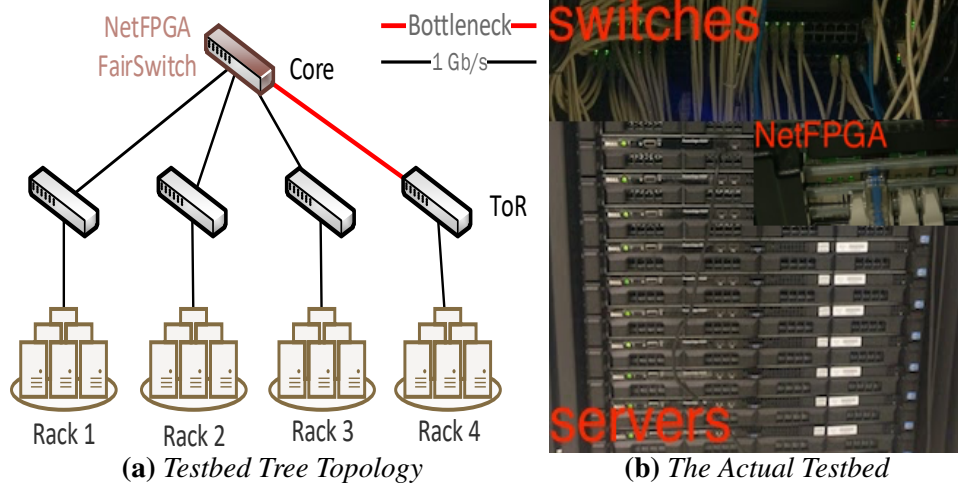


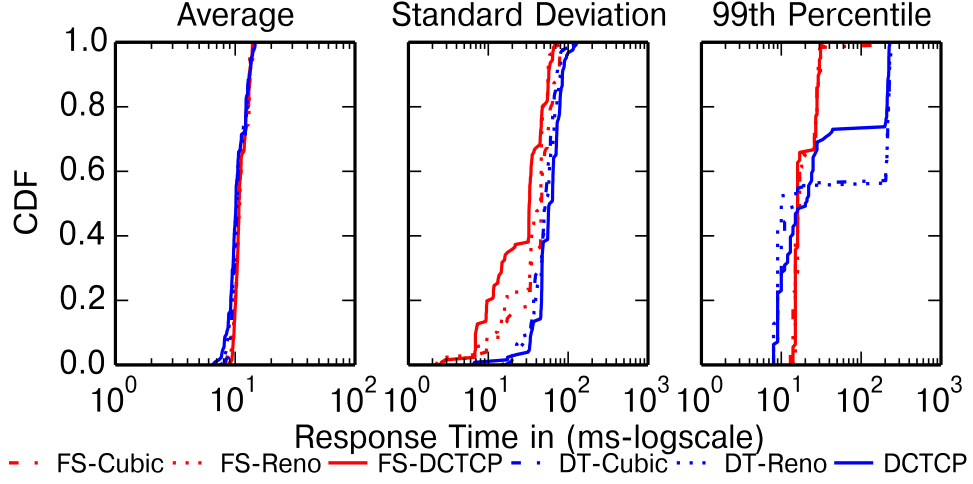
Figure 5: Testbed topology for FairSwitch evaluation

webpage **"index.html"** of size 11.5KB for creating mice flows. We setup different scenarios to reproduce both "incast" and "incast with elephants" situations on the bottleneck link connected to receiving rack 4. To emulate virtual guests and increase the number of senders dramatically, senders are created by creating multiple virtual ports at the end-hosts on open vswitch and binding an iperf process or an Apache client/server process to each virtual port. This allows us emulate traffic originating from any number of VMs to simplify the creation of scenarios with a large number of flows in the network. The objectives are: *i)* verify whether FairSwitch is able to support more TCP connections and maintains high link utilization; *ii)* verify the ability of FairSwitch to equally and fairly allocate bandwidth among conflicting mice and elephant TCP flows; *iii)* verify by how much FairSwitch is able to improve the FCT of the time-critical mice flows.

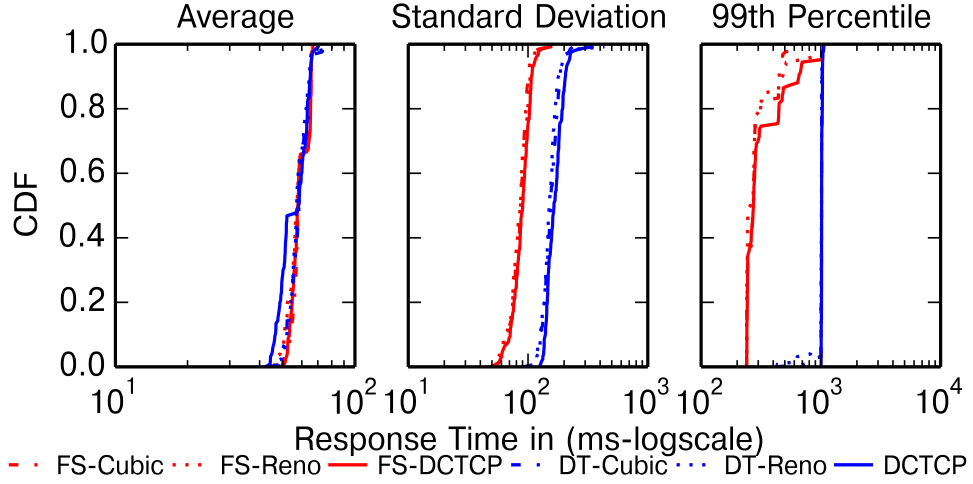
2.3 Hardware Experimental Results

Incast Scenario without Background Traffic: we run two incast mildly and heavily loaded scenarios where a large number of mice flows request a large content divided into 11.5KB chunks.

Experimental Setup: In both scenarios, each of the 7 servers in rack 4, issue web requests via the Apache benchmark tool [4] requesting **"index.html"** page of size 11.5KB from each of the 21 servers in rack 1, 2 and 3. The requests exclude web servers listening on the other 3 ports of the same physical machine



(a) Medium case: 126 concurrent mice flows retrieving 1.45Gbytes



(b) Heavy case: 630 concurrent mice flows retrieving 7.25Gbytes

Figure 6: Average, Standard Deviation and Max FCT for IncastGuard vs (TCP with DropTail or DCTCP): each flow requests 11.5MB file (divided into 1000 11.5KB blocks)

of the requester, hence a total of 126 ($21 \times 7 - 3 \times 7$) synchronized requests are issued. In the mildly loaded scenario, each request is repeated a thousand consecutive times by Apache benchmark which is equivalent to an 11.5 MByte file for

each requester ($11.5 \times 126 \approx 1.5$ GBytes total transfer through the bottleneck link). In the heavily loaded case, we repeat the same experiment with a thousand consecutive requests however, we use now 5 parallel TCP connections instead of just 1, which are the same amount of transfer through the bottleneck link within the same period but with more concurrent connections compared to the mild load case. Note that, Apache benchmark, at the 0^{th} sec, starts requesting the web page 1000 or 2000 times then it reports different statistics over the total requests.

Experimental Results: Figure 9 shows that, under heavy load, FairSwitch achieves a significantly improved performance for TCP flows in terms of the FCT variation from average and the FCT of the tail-end flows. Even though RWNDQ still improves both metric in the mild case, the performance gains are not sound like the heavy scenario. The competing mice flows benefit under FairSwitch in the mild case by achieving almost the same FCT on average but with an order-of-magnitude smaller standard deviation compared to TCP (Cubic, Reno) with DropTail and DCTCP. In addition, it can improve the FCT of the end of the tail (i.e., the maximum FCT) by two orders-of-magnitude suggesting that almost all flows (including tails) can meet their deadlines. However, it can not achieve the same improvement under heavy load because in this case congestion happens immediately at the onset of communication. That is, with 630 flows each with 1 MSS (1460 bytes) of window the total traffic of 920 KBytes is $\approx 3.2\%$ larger than the size of the bottleneck pipe or, 287KB which is the switch Buffer size plus the bandwidth delay product. Finally, FairSwitch can help incast traffic grab their fair-share out of the network-hogging elephants, Figure 7 shows that, it can significantly decrease the drop rate during incast events by $\approx 4000\%$ in medium load compared to only $\approx 300\%$ in heavy load scenarios.

Low frequency Incast with Background Traffic: In the following experiments, we run a medium load scenario where mice flows compete with elephants flows under a bloated buffer situation. Our goal is to see if FairSwitch can help mice flows grab some bandwidth from elephants and to see the resulting and effects on the throughput of elephants.

Experimental Setup: We first generate 21 synchronized iperf [5] elephant flows from each server in racks 1, 2 and 3 towards rack 4 through the bottleneck link continuously sending for 20 secs. Then, we again invoke Apache-benchmark on the servers of rack 4 to request **"index.html"** from each of the web servers running on the servers in racks 1, 2 and 3. Hence these web request must compete for the bottleneck bandwidth with each other and most importantly with iperf elephant traffic. A single incast epoch of Web requests is scheduled to run for 100 consecutive requests (i.e., each client requests a 1.15MB file partitioned into 100

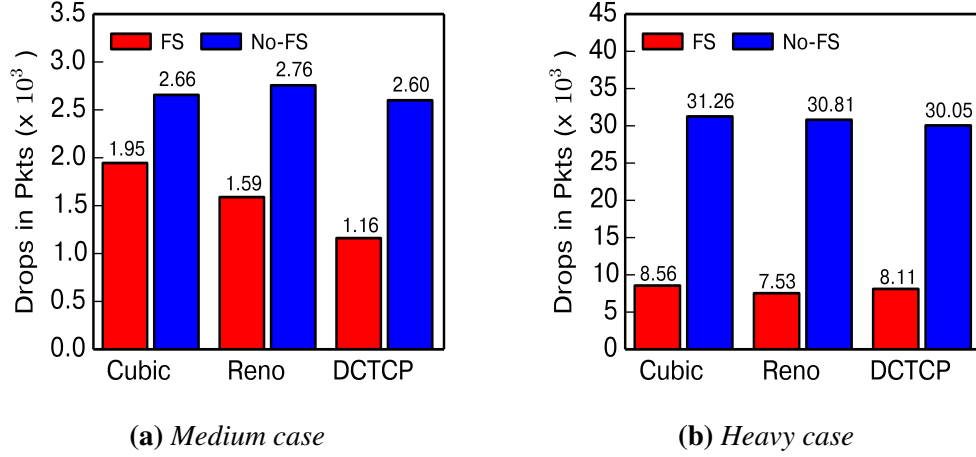


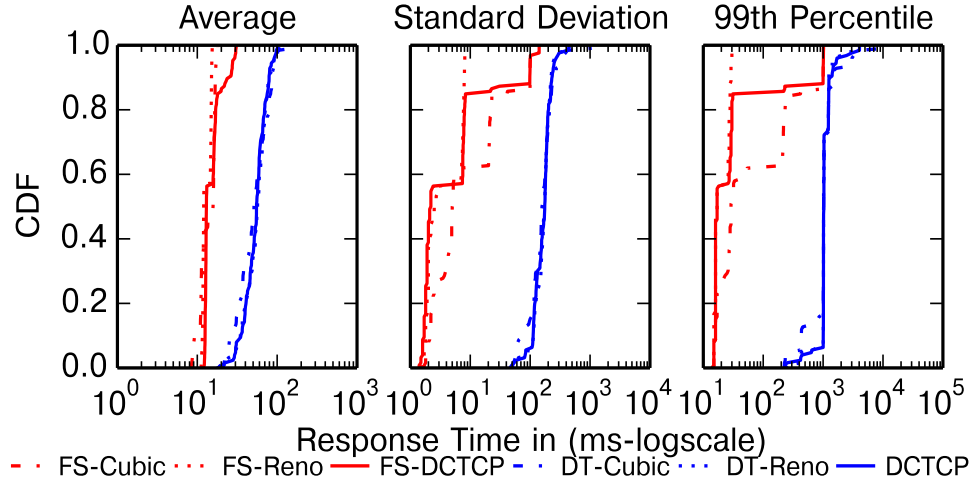
Figure 7: Total Packet Drops during the experiment: TCP assisted by IncastGuard vs (TCP with DropTail or DCTCP)

11.5KB chunks totaling ≈ 145 MBytes) after elephants have reached steady state (i.e., at the 10th sec) and then different statistics are reported.

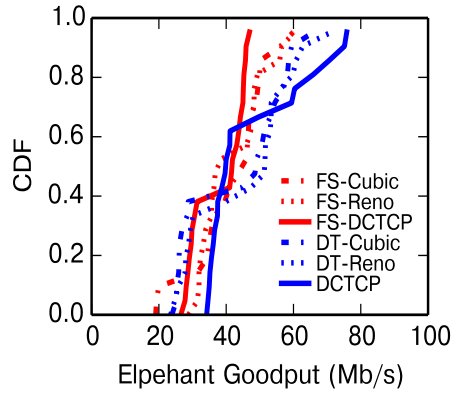
Experimental Results: Figure shows that, in medium load, FairSwitch achieves a FCT improvements for mice while nearly not affecting elephants' performance. Mice flows benefit with FairSwitch by improving on FCT on average and again with one order-of-magnitude reduction in FCT standard deviation compared to TCP (Cubic, Reno) with DropTail and DCTCP. Also, in terms of the tail (i.e., the last and similarly the 99%), FairSwitch reduces the tail FCT by two order-of-magnitude almost close to the average and within 10's of ms. The improvement means mice flows finish quickly within their deadlines. Fig 8b shows that elephant flows are almost not affected by FairSwitch intervention by throttling their rates during the short incast periods. In Figure 8c, the drops under FairSwitch is reduced by its efficient rate control during incast hence mice flows avoid long timeouts of at least 200ms.

High frequency Incast with Background Traffic: We repeat the above experiment, increasing the frequency of Mice incast epochs to 9 times within the 20 second period (i.e., at the 2nd, 4th, ..., and 18th sec). In each epoch, each server requests a 1.15MB file partitioned into 100 11.5KB chunks totaling ≈ 145 MBytes per epoch and a total of ≈ 1.3 GBytes for all 9 epochs. As shown in

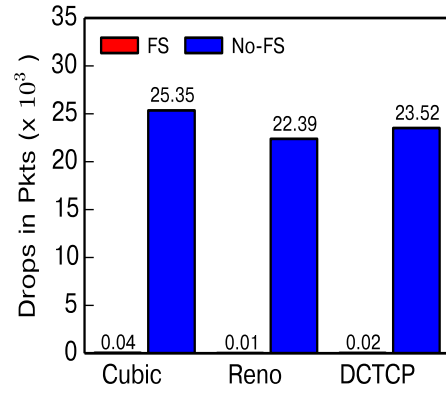
Experimental Results: Figure 9a, even with the increased incast frequency, FairSwitch is still able to keep up with higher incast event rates even-though mice



(a) 126 concurrent mice flows retrieving 1.45Gbytes one time only



(b) Average elephant throughput

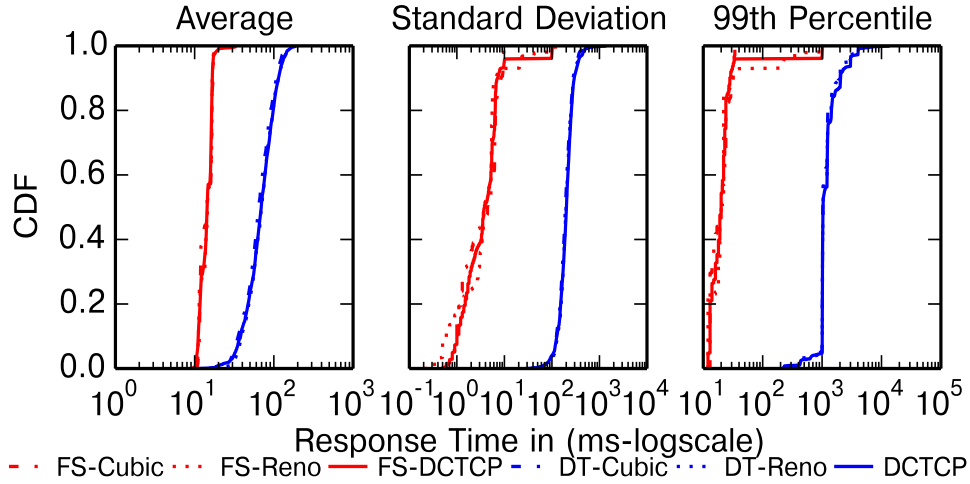


(c) Total packet drops

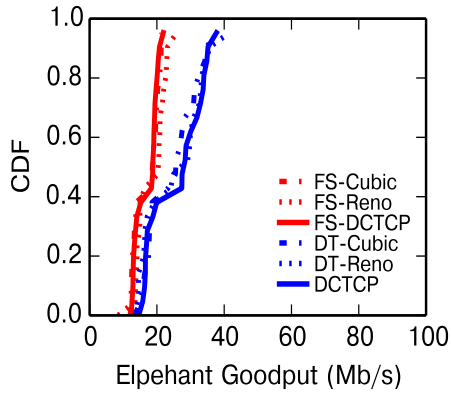
Figure 8: Performance of FairSwitch vs (TCP with DropTail or DCTCP): each of the 126 mice flow requests 1.15 MB file (divided into 100 11.5KB blocks) 1 time while competing with 21 elephants

flows are also fighting their way against fat elephant flows. Mice flows' average and standard deviation of FCT see similar improvement as the previous experiments compared to TCP with DropTail and DCTCP. This can be attributed to the decreased packet drops rate with the help of FairSwitch and hence lesser chances of experiencing timeouts as show in Figure9c. Compared to the previous ex-

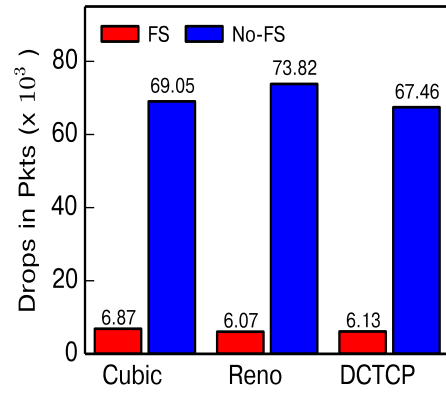
periment, Figure 9b shows that, elephants throughput is reduced because of the frequent rate throttling introduced by FairSwitch during incast periods. However, we believe that the bandwidth is fairly utilized by mice and elephants with Fair-Switch, hence the lower achieved average by elephants when Mice are active in the network.



(a) 126 concurrent mice flows retrieving 145MB for 9 epochs



(b) Average elephant throughput



(c) Total packet drops

Figure 9: Performance of FairSwitch vs (TCP with DropTail or DCTCP): each of the 126 mice flow requests 1.15MB file (divided into 100 11.5KB blocks) 9 times while competing with 21 elephants

In summary the experimental results of the hardware prototype support and highlight the performance gains especially for OLDI applications obtained by adopting FairSwitch system. In particular, they show that:

- FairSwitch minimizes the variance of mice flow completion times and significantly reduce by 2 orders-of-magnitude the FCT for the tail end.
- FairSwitch can maintain the same improvement even if the bandwidth-hungry elephants are hogging the network.
- FairSwitch efficiently handles incast epochs, in low and high frequency, by allowing mice to compete in a fair game against elephants.
- FairSwitch fulfilled its requirements with no more than default assumptions about the network stack and without imposing any modifications to tenants' VMs such as a new TCP.

3 IncastGuard for Incast mitigation in Data Centers

In this section, we further present the hardware design aspects of IncastGuard in NetFPGA platform. IncastGuard is based on the following ideas: 1) In [1], we discussed the Incast problem and hence we concluded that to predict and circumvent the consequences of the incast congestion resulting from synchronized TCP flows, a mechanism has to intervene to empty the buffer space for these flows to pass in the network peacefully; 2) The switches are found to be as the right place to perform Incast predication. Hence, IncastGuard is proposed as a switch prototype of "IQM"; 3) The ultimate-objective of this approach is to avoid modifying existing protocols at the sender and receiver. Typically, IncastGuard relies only on universally accepted mechanisms for congestion notification (i.e., Flow Control). The switch intervenes temporarily and can actively rewrite the *Rwnd* field as a means to throttling the sender's rate of each connection. 4) How frequently should such rewriting take place? in this case, the switch rewrites *Rwnd* for every ACK on the backward path to the senders **only if Incast flag is ON**.

3.1 NetFPGA Design and Implementation

Again, the hardware prototype of IncastGuard was developed on the NetFPGA-1G card using "Verliog" hardware description language. Similar to FairSwitch,

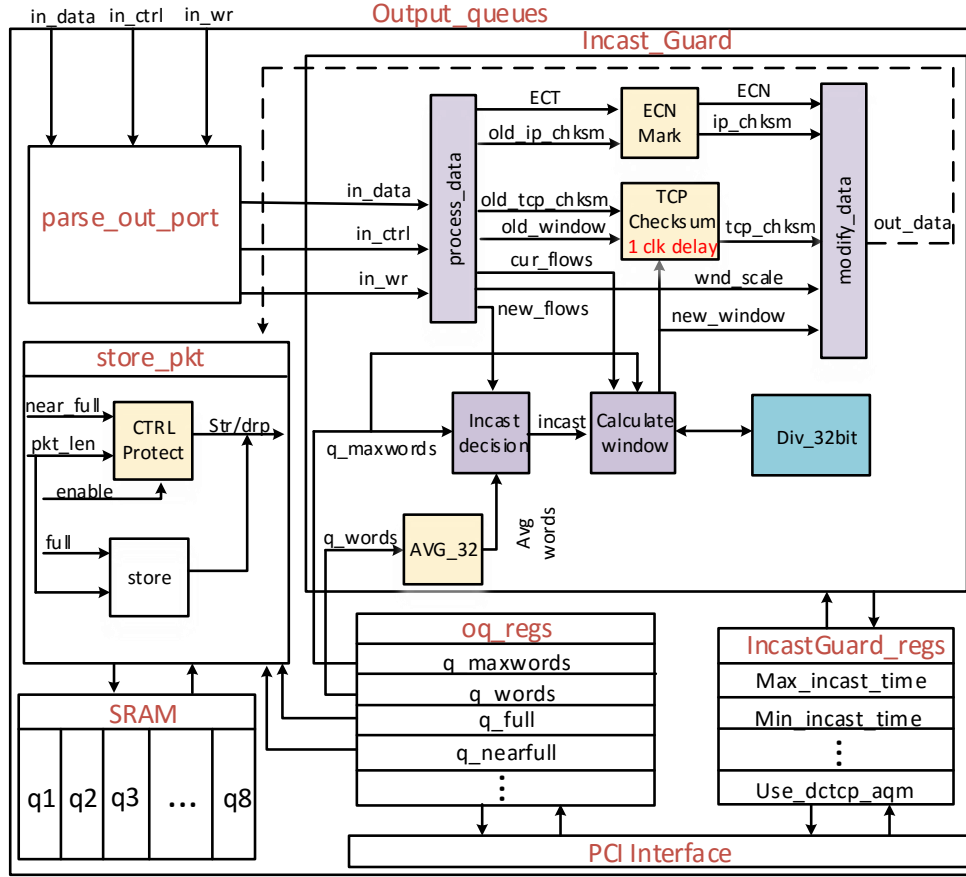


Figure 10: *IncastGuard NetFPGA-based system design*

we have chosen to build it on top of the reference switch by inserting the Incast_Guard module into the processing pipeline of “output queues” module. The required modifications were realized through 600 lines of Verilog code including declarations.

3.1.1 NetFPGA IncastGuard module

“Incast_Guard” module which is instantiated within the top “output_queues” module as shown in Fig 10. The module itself instantiates a number of helper modules namely “avg_32”, “div_32”, “TCP_checksum” and “ECN_mark” for different data processing. These modules were presented in the previous Section 2.

The module keeps track of separate per-output-queue counters for SYN/SYN-

ACK, FIN/FIN-ACK and RST packets seen going through the queue. This information can be extracted by applying the state machine on `in_data` to process the headers. In each period, the newly active flows is tracked for each of the port queues. Each time a SYN-ACK is received, the relative SYN flow counter is incremented by 1 for the incoming and the outgoing queues. And whenever a FIN-ACK/RST is received, FIN is decreased by one for the outgoing direction of the FIN-ACK/RST. At any time, the absolute difference between SYNs and FINs represents the number of new flows in this round. Incast flag state is updated from OFF to ON if the extra traffic plus the queue occupancy would exceed the preset limit. The flag is updated from ON to OFF if the queue occupancy goes below the preset safe threshold.

On the arrival of each ACK, before it is forwarded to a certain output queue (port), the receive window field of TCP header is updated with the parameter Incast window (i.e., 1 MSS) whenever the flag is ON. This window is right shifted by the “window scale” value which can be extracted out from the reserved-bits field shown in Figure 2. Note that, the `safe_threshold` should be set in a way to preserve an almost empty queue for absorbing packet bursts and large enough to keep the link fully utilized.

Similar to FairSwitch, a new helper module is used, namely a **IncastGuard_regs** module. This module is instantiated in the `output_queues` module to store all IncastGuard related parameters (stored in registers) for usage by various module functions. The FairSwitch parameters can read and/or set via read/write over PCI interface.

Figure 10 shows how IncastGuard module interacts with the other modules within `output_queues` main module. Most interactions are similar to FairSwitch except for the inner operations performed by the IncastGuard module.

The complexity of our module is assessed in terms of resource usage, Table 2 highlights the increase in the resources between the reference switch design and the modified one that includes IncastGuard module.

3.2 Deployment of the Hardware Prototype

We now investigate the performance of the hardware prototype of IQM namely “IncastGuard” switch as well as its end-host helper module which are presented in Appendix ???. Figure 11 shows the deployment of IncastGuard system and how would its components interact with each other. It also shows that the module resides right above the NIC driver for a non-virtualized setups and right below the hypervisor to support VMs in cloud datacenters. Hence, this placement does not

Table 2: Comparison of resource usage between the reference switch and IncastGuard

	Reference Switch		IncastGuard		NetFPGA Total
	Total	Percentage	Total	Percentage	
Slices	9605	40%	14551	61%	23616
Slice Flip Flops	8364	17%	12481	26%	47323
4 input LUTs	14067	29%	22610	47%	47232
BRAMs	22	9%	22	9%	232
Bonded IOBs	360	52%	360	52%	692
CGLKs	8	50%	8	50%	16
DCM	6	75%	6	75%	8

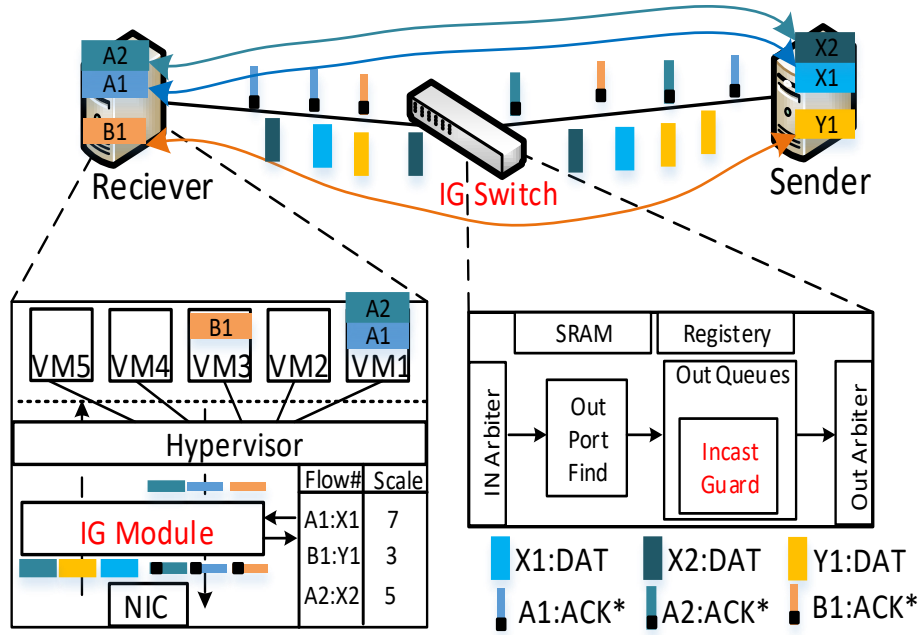


Figure 11: IncastGuard System: It consists of an end-host module that attaches scaling value to ACKs and IncastGuard switch which performs incast detection and mitigation functions.

touch any network stack implementation of host or guest OS, making it readily deployable in production datacenters. The system performs the following functions:

1. At connection-setup, flows are hashed into a hash-table with the flow's 4-

tuples (i.e., source IP, dest. IP, source port and dest. port) used as the key and the scale used as the value.

2. Flow entries are cleared from the table when a connection is closed (i.e., FIN is sent out).
3. The module writes the scale factor for all outgoing ACK packets in the 4-bit reserved field of TCP headers (or, encode the scale factor into 4-bits of the receive window field and use the remaining 12 bits for the receive window values).
4. The end-host module tracks the scaling factor used by local communicating end-points and explicitly append this information only to outgoing ACKs of the corresponding flow.
5. The switch whenever it detects the onset of possible incast event for one of the ports, it immediately switches to incast mode and will start window updates in the incoming ACKs.
6. IncastGuard uses the attached scale factor to rescale the used window so that it can be interpreted correctly by the ACK receiving end-point. Then, it clears the used reserved bits to avoid packet being dropped by destination due to invalid TCP check-sum value which avoids the need for recalculating TCP checksum at the end-host and the switch.

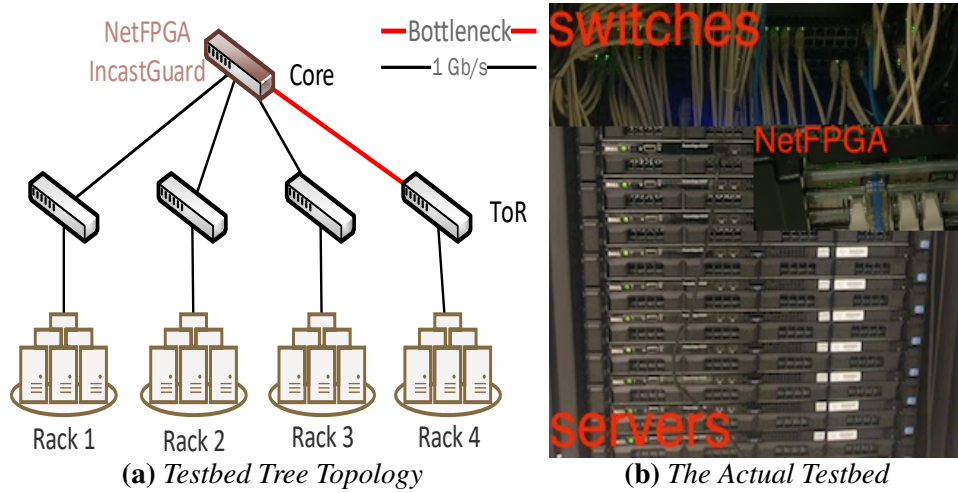


Figure 12: Testbed topology for IncastGuard evaluation

3.3 Hardware Experimental Results

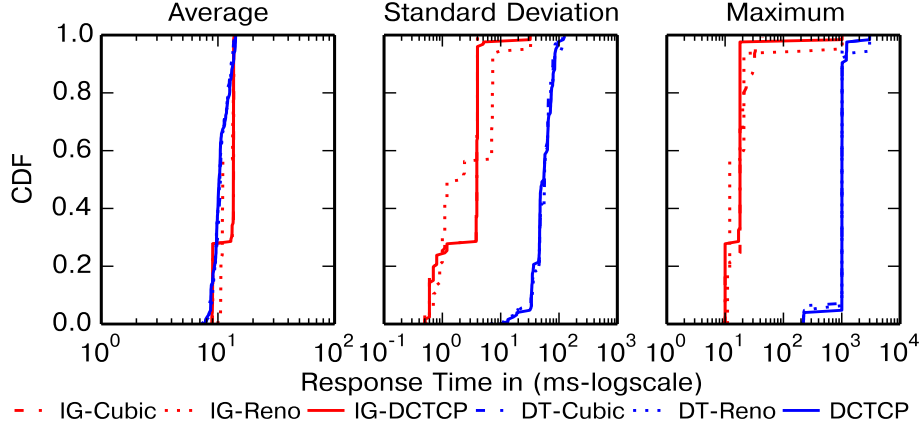
In this set of experiments, we deploy the NetFPGA-based IncastGuard switch in a small-scale real-testbed with same configuration and setup of FairSwitch deployment in Section ?? . Figure 12 shows the topology used in the experiments and Figure 12b shows a photo taken of the real-life deployment. The objectives are: *i)* Does IncastGuard help TCP support many more connections and maintains high link utilization; *ii)* Does IncastGuard help with reducing incast congestion effect on TCP flows; *iii)* Does IncastGuard improve the FCT of mice when they competing for the bottleneck link with elephants.

Incast Scenario without Background Traffic: we run two low and high frequency incast scenarios where a large number of mice flows request a large content divided into 11.5KB chunks.

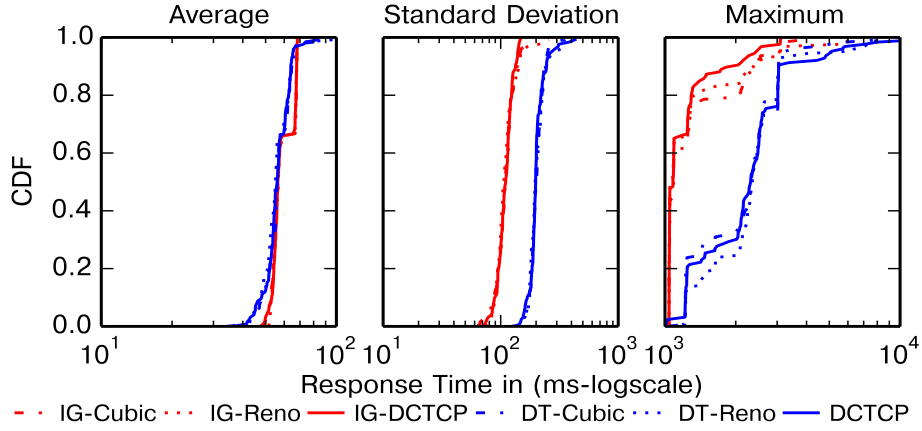
Experimental Setup: in these scenarios, all 7 servers in rack 4 issue Apache benchmark requests to get "index.html" page from each of the 21 servers in rack 1, 2 and 3. A total of 126 ($21 \times 7 - 3 \times 7$) synchronized requests are issued. In the low frequency scenario, the request is issued a thousand consecutive times which results into to a total of 11.5 MBytes transfer for each requester (i.e., total transfer of $11.5MB \times 126 \approx 1.5GB$ through the bottleneck link). In the high frequency case, 5 parallel TCP connections is used to mimic a more heavy incast of 630 flows.

Experimental Results: Figure 16 shows that, in both scenarios, IncastGuard achieves a significantly improved performance for TCP flows. The competing mice flows benefit under IncastGuard in the mild case by achieving almost the same FCT on average but with an order-of-magnitude smaller standard deviation compared to TCP (Cubic, Reno) with DropTail and DCTCP. In addition, it can improve the FCT of the end of the tail (i.e., maximum FCT) by two orders-of-magnitude suggesting that almost all flows (including tail-ends) can meet their deadlines. However, it can not achieve the same improvement under heavy load because in this case congestion happens immediately at the onset of communication. That is, with 630 flows each with 1 MSS (1460 bytes) of window the total traffic of 920 KBytes is $\approx 3.2\%$ larger than the size of the bottleneck pipe or, 287KB which is the switch Buffer size plus the bandwidth delay product. Finally, IncastGuard can efficiently detect the incast and proactively throttle the flows to avoid packet drops, Figure 14 shows that, it can significantly decrease the drop rate during incast events by $\approx 4000\%$ in medium load compared to only $\approx 300\%$ in heavy load scenarios.

Incast Scenario with Background Traffic: we run a medium frequency in-



(a) *Medium case: 126 concurrent mice flows retrieving 1.45Gbytes*



(b) *Heavy case: 630 concurrent mice flows retrieving 7.25Gbytes*

Figure 13: Average, Standard Deviation and Maximum FCT for IncastGuard vs (TCP with DropTail or DCTCP): each flow requests 11.5MB file (divided into 1000 11.5KB blocks)

cast scenario but mice flows have to compete with elephants to a buffer-bloating situation. Our goal is to see if IncastGuard can help mice flows grab some bandwidth from elephants and to observe how it may affect the throughput of elephants.

Experimental Setup: we first generate 21 synchronized iperf [5] elephant flows from each server in racks 1, 2 and 3 towards rack 4 through the bottleneck link continuously sending for 20 secs. Then, We again invoke Apache-benchmark on the servers of rack 4 to request **"index.html"** from each of the web servers

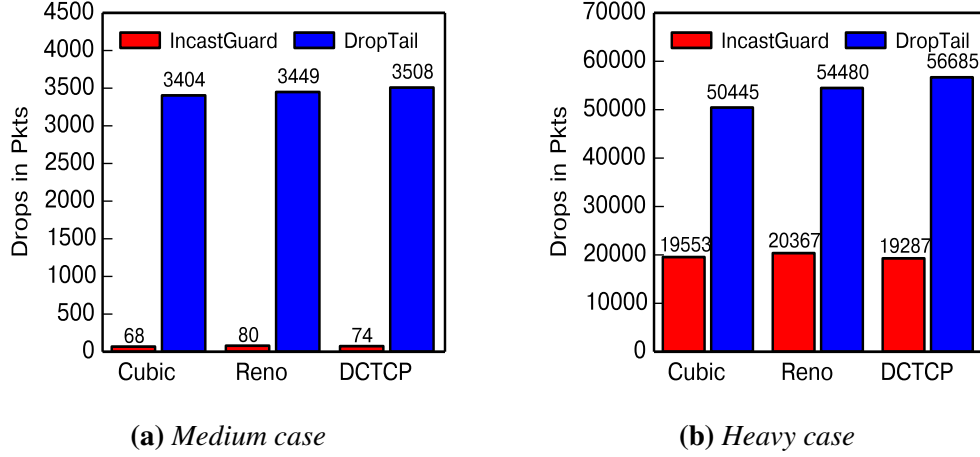
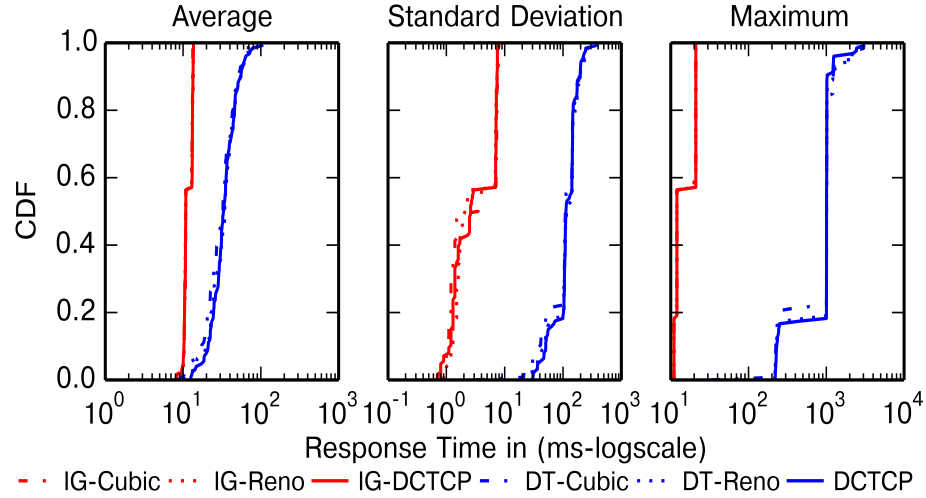


Figure 14: Total Packet Drops during the experiment: TCP assisted by IncastGuard vs (TCP with DropTail or DCTCP)

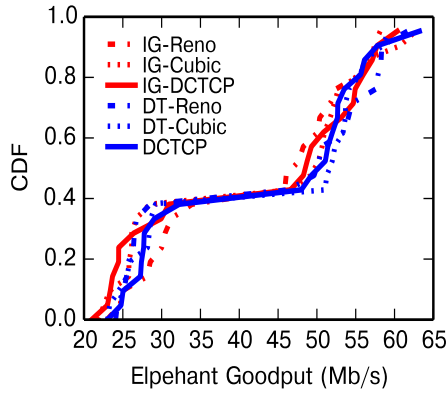
running on the servers in racks 1, 2 and 3. Hence these web request must compete for the bottleneck bandwidth with each other and most importantly with iperf elephant traffic. A single incast epoch of Web requests is scheduled to run for 100 consecutive requests (i.e., each client requests a 1.15MB file partitioned into 100 11.5KB chunks totaling ≈ 145 MBytes) after elephants have reached steady state (i.e., at the 10th sec) and then different statistics are reported.

Experimental Results: Figure shows that, in medium load, IncastGuard achieves significant FCT improvements for mice while nearly not affecting elephants' performance. Mice flows benefit with IncastGuard by improving on FCT on average and again with one order-of-magnitude reduction in FCT standard deviation compared to TCP (Cubic, Reno) with DropTail and DCTCP. Also, in terms of the tail (i.e., the last and similarly the 99%), IncastGuard reduces the tail FCT by two order-of-magnitude almost close to the average and within 10's of ms. The improvement means mice flows finish quickly within their deadlines. Fig 15b shows that elephant flows are almost not affected by IncastGuard's intervention by throttling their rates during the short incast periods. In Figure 15c, the drops under IncastGuard is reduced by its efficient rate control during incast hence mice flows avoid long timeouts of at least 200ms.

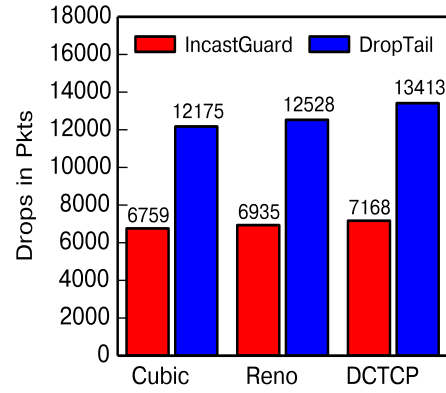
We repeat the above experiment, increasing the frequency of Mice incast epochs to 9 times within the 20 second period (i.e., at the 2nd, 4th, ..., and 18th sec). In each epoch, each server requests a 1.15MB file partitioned into 100 11.5KB



(a) 126 concurrent mice flows retrieving 1.45Gbytes one time only



(b) Average elephant throughput



(c) Total packet drops

Figure 15: Performance of IncastGuard vs (TCP with DropTail or DCTCP): each of the 126 mice flow requests 1.15MB file (divided into 100 11.5KB blocks) 1 time while competing with 21 elephants

chunks totaling $\approx 145MB$ per epoch and a total of $\approx 1.3GB$ for all 9 epochs. As shown in Figure 16a, even with the increased incast frequency, IncastGuard is still able to keep up with higher incast event rates even-though mice flows are also fighting their way against fat elephant flows. Mice flows' average and standard deviation of FCT see similar improvement as the previous experiments compared

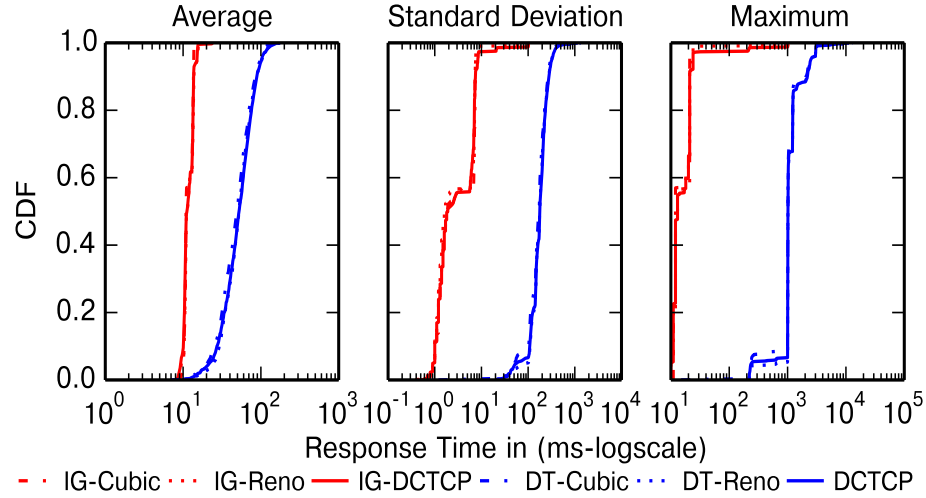
to TCP with DropTail and DCTCP. This can be attributed to the decreased packet drops rate with the help of IncastGuard and hence lesser chances of experiencing timeouts as show in Figure 16c. Compared to the previous experiment, Figure 16b shows that, elephants throughput is reduced because of the frequent rate throttling introduced by IncastGuard during incast periods. However, We believe that the bandwidth is fairly utilized by mice and elephants with IncastGuard, hence the lower achieved average by elephants when Mice are active in the network.

3.3.1 Backward Path Pressure via Cross Traffic:

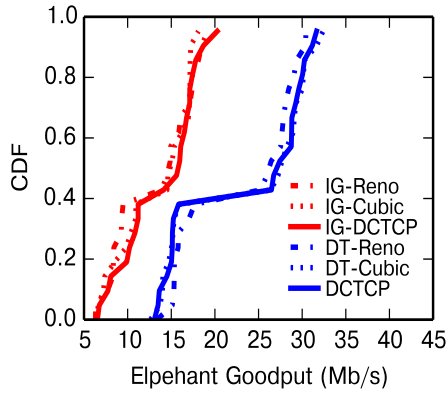
previously ACK (or control packets) protection was disabled, hence we enable this feature and run an experiment that involves cross (e.g., backward) traffic to pressure the backward path of ACKs. To this end, we first generate 21 backward iperf flows until the end of the experiment from each server in racks 4, 3 and 1 towards rack 2 to pressure ACKs forwarded to link 2 (i.e., rack 2). Shortly after 3 secs, 21 synchronized forward elephant flows from each server in racks 1, 2 and 3 towards rack 4 are started for 30 secs. At the 15th sec, Apache-benchmark on rack 4 is used to start mice flows to compete for the bottleneck against the elephants. This results in an incast epoch of 100 consecutive requests from each of the 126 server (6×21). Figure 17 shows that, with control protection, an improvement for the affected “rack 2” requests of $\approx 5 - 10\%$ on average, standard deviation and the tail end compared to IncastGuard with no protection. Interestingly, the percentage of failed Apache-benchmark tests dropped from 6.4% in both not protected IncastGuard and not-assisted cubic cases to 1.5% when control protection was applied with any amount of buffer room reservation. The results suggest that a room of $\leq 6.25\%$ of buffer space is sufficient because there is no noticeable improvement for any extra buffer reserved above the 6.25% threshold.

In summary the experimental results of the hardware prototype support and highlight the performance gains especially for OLDI applications obtained by adopting IncastGuard system. In particular, they show that:

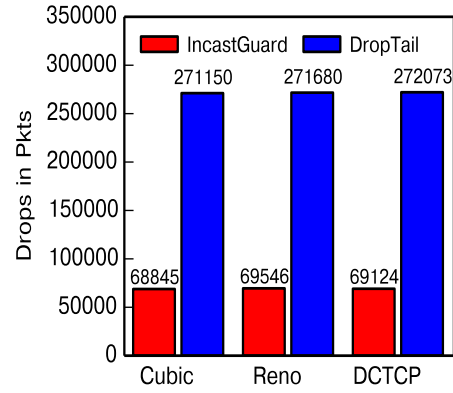
- It minimizes the variance of mice flow completion times and significantly reduce by 2 orders-of-magnitude the FCT for the tail end.
- It maintains its improvements even if the bandwidth-hungry elephants are hogging the network.
- It efficiently handles incast epochs, in low and high frequency, by allowing mice to compete in a fair game against elephants.



(a) 126 concurrent mice flows retrieving 145MB for 9 epochs



(b) Average elephant throughput



(c) Total packet drops

Figure 16: Performance of IncastGuard vs (TCP with DropTail or DCTCP): each of the 126 mice flow requests 1.15 MB file (divided into 100 11.5KB blocks) 9 times while competing with 21 elephants

- It fulfilled its requirements with no more than default assumptions about the network stack and without imposing any modifications to tenants' VMs such as a new TCP.

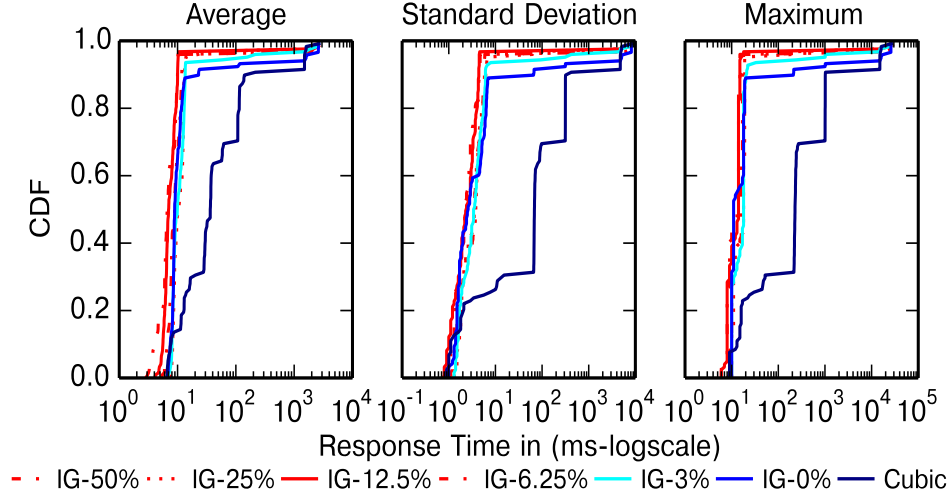


Figure 17: *IncastGuard with control packets protection*

4 HystSwitch for Hysteresis-Based Switching

In this section, we discuss the design aspects of HystSwitch which is based on the following ideas: 1) The reasoning in [8] calls for a switch scheme that is able to stretch out the TCP loss cycle ultimately to improve the FCT of TCP flows; 2) The switches have to switch among normal TCP and slow CBR operation. Hence, HystSwitch adopts an AQM namely “HSCC” to switch among TCP and CBR on a counter-clockwise hysteresis basis; 3) The switch will be involved in actively rewriting the *Rwnd* field to control sender rates of TCP flows. 4) How frequently should such rewriting take place? in this case, the switch rewrites *Rwnd* for every ACK on the backward path to the senders **only when the hysteresis control is in the ON state**.

HystSwitch is a hardware implementation of the HSCC algorithm in [8]. HSCC aims to improve short TCP performance by reducing the loss probability and hence the RTO. This is accomplished via means of expanding the TCP loss cycles of the typical congestion avoidance saw-toothed steady state of TCP.

4.1 NetFPGA Design and Implementation

HystSwitch was developed using “Verliog” language on the NetFPGA-1G platform. It leverages the reference switch and its module is integrated into the main

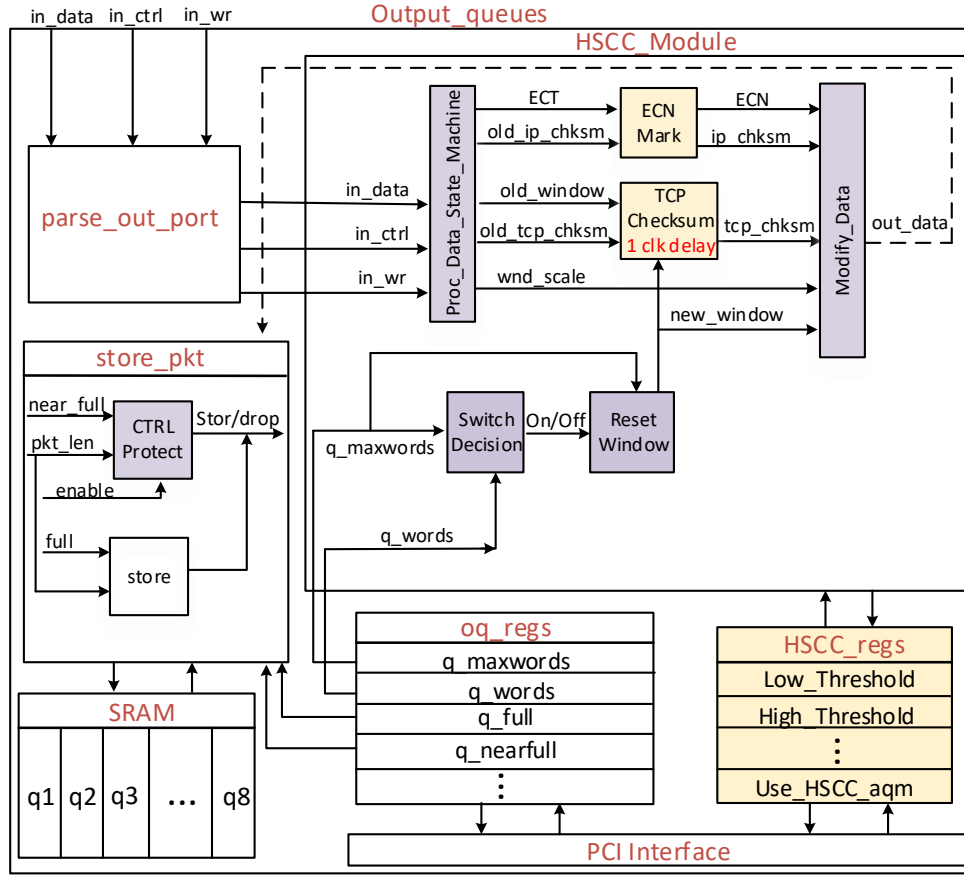


Figure 18: HystSwitch NetFPGA-based system diagram

“output queues” module of the switch. The required modifications were realized through 430 lines of Verliog code including declarations.

4.1.1 NetFPGA FairSwitch module

The main design is implemented in “Hyst_Switch” module which is instantiated within the top “output_queues” module as shown in Fig 18. The Hyst_Switch module instantiates same helper modules namely “TCP_checksum” and “ECN_mark” for TCP checksum recalculation and DCTCP ECN marking functions, respectively.

In Figure 18, “HSCC.Module” module keeps track of the current queue occupancy to determine whether it is above or below the high and low threshold.

The hysteresis control state is switched ON/OFF by the “Switch Decision” element. The value is updated each time the high or low thresholds are crossed from below and above, respectively. The reset window (i.e., resetting back to 1 MSS) is decided by the “Reset Window” element. When reset window is active, new window value is used to update the receive window field and TCP checksum is recalculated.

On the arrival of each ACK, before it is forwarded to a certain output queue (port), the receive window field of TCP header is updated (only if it is less than the current value in TCP header) with the newly calculated window. The new window is right shifted by the “window scale” value which can be extracted out from the reserved-bits field shown in Figure 2.

Figure 18 shows similar interactions between HSCC module and the other modules in the switch to the ones presented for FairSwitch in Section 2.

The complexity of our module in terms of resource usage is presented in Table 3. It highlights the increase in the resources between the reference switch design and the modified switch which instantiates our HystSwitch module.

Table 3: Comparison of resource usage between the reference switch and HystSwitch

	Reference Switch		HystSwitch		NetFPGA Total
	Total	Percentage	Total	Percentage	
Slices	9605	40%	12038	50%	23616
Slice Flip Flops	8364	17%	9640	20%	47323
4 input LUTs	14067	29%	18326	38%	47232
BRAMs	22	9%	22	9%	232
Bonded IOBs	360	52%	360	52%	692
CGLKs	8	50%	8	50%	16
DCM	6	75%	6	75%	8

5 Report Summary

In this report, we have covered the implementation and prototyping of the NetFPGA designs which were used for the synthesis of our switch-based schemes. NetFPGA is an excellent platform for prototyping and testing of proposed hardware changes in the networking devices (e.g., NIC, switch, router). Before NetFPGA, such changes for research parties were limited to simulations but now we

see NetFPGA as a framework that would encourage more research that involve changes in the networking hardware.

References

- [1] A. M. Abdelmoniem and B. Bensaou. Incast-Aware Switch-Assisted TCP congestion control for data centers. In *IEEE Global Communications Conference (GlobeCom)*, 2015.
- [2] A. M. Abdelmoniem and B. Bensaou. Reconciling mice and elephants in data center networks. In *IEEE International Conference on Cloud Networking (CloudNet)*, 2015.
- [3] M. Alizadeh. Data Center TCP (DCTCP). <http://simula.stanford.edu/alizade/Site/DCTCP.html>.
- [4] Apache.org. Apache HTTP server benchmarking tool. <http://httpd.apache.org/docs/2.2/programs/ab.html>.
- [5] iperf. The TCP/UDP Bandwidth Measurement Tool. <https://iperf.fr/>.
- [6] V. Jackbson, R. Braden, and D. Borman. TCP Extensions for High Performance, 1992. <https://www.ietf.org/rfc/rfc1323.txt>.
- [7] J. W. Lockwood, N. McKeown, G. Watson, G. Gibb, P. Hartke, J. Naous, R. Raghuraman, and J. Luo. NetFPGA - An open platform for gigabit-rate network switching and routing. In *Proceedings of IEEE International Conference on Microelectronic Systems Education (MSE)*, pages 160–161, 2007.
- [8] A. M. Abdelmoniem and B. Bensaou. Improving tcp performance in data centers via hysteretic control. 2017.
- [9] NetFilter.org. NetFilter Packet Filtering Framework for linux. <http://www.netfilter.org/>.
- [10] netfpga.org. NetFPGA 1G Specifications. http://netfpga.org/1G_specs.html.
- [11] A. Rijssinghani. RFC 1624 - Computation of the Internet Checksum via Incremental Update, 1994. <https://tools.ietf.org/html/rfc1624>.